GUÍA PRÁCTICA

Angular y fundamentos web

HTTP, DOM, CSS y métodos GET, POST, PUT, PATCH y DELETE explicados con el código real del proyecto HelpDesk.

Proyecto	Qué usarás para aprender
HelpDesk (carpeta `helpdesk/`)	Angular 22 + FastAPI + PostgreSQL + JWT + roles + correos
StoreAPI (carpeta `StoreAPI/`)	HTML, CSS y JavaScript puro consumiendo una API pública

Contenido

1. Cómo funciona la web 2

2. HTTP: pedir y responder 3

3. Métodos HTTP: GET, POST, PUT, PATCH, DELETE 4

4. HTML y el DOM 6

5. CSS: los estilos 7

6. Angular paso a paso 9

7. Autenticación: JWT, interceptor y roles 12

8. Todo junto: la vida de un clic 14

9. Ejercicios y glosario 15

> **Cómo usar esta guía**
> Cada concepto se explica y luego se muestra **dónde aparece en el proyecto**. Abre los archivos mencionados, ejecuta la app y mira la pestaña **Network** de las herramientas de desarrollo (F12) mientras haces clic: verás en vivo todo lo que aquí se describe.

CAPÍTULO 1

Cómo funciona la web

Toda aplicación web, sea una tienda o un helpdesk, se reduce a una conversación entre dos programas: el **cliente** (tu navegador) y el **servidor**. El cliente pide cosas y el servidor responde.

En HelpDesk hay **tres** piezas hablando entre sí:

Pieza	Dónde corre	Responsabilidad
Frontend (Angular)	En tu navegador, `localhost:4200`	Mostrar pantallas y capturar acciones del usuario.
Backend (FastAPI)	Contenedor Docker, `localhost:8000`	Reglas de negocio, seguridad, validación.
Base de datos (PostgreSQL)	Contenedor Docker, red interna	Guardar usuarios y tickets de forma permanente.

El navegador **nunca** habla directamente con la base de datos. Eso sería un agujero de seguridad: cualquiera podría leer o borrar todo. El backend es el único guardián.

Anatomía de una URL

```
http://localhost:8000/api/tickets?priority=high#top
└─┘   └────────┘└─┘└──────────┘└──────────────┘└─┘
protocolo  host   puerto   ruta      query     fragmento
```

- **Protocolo**: `http` (sin cifrar) o `https` (cifrado). En producción siempre https.
- **Host y puerto**: a qué máquina y a qué programa dentro de ella. Por eso HelpDesk usa 4200, 8000 y 8025.
- **Ruta**: el recurso. `/api/tickets` significa "la colección de tickets".
- **Query**: filtros opcionales tras el `?`.
- **Fragmento**: lo usa solo el navegador; nunca llega al servidor.

JSON: el idioma de las APIs

Frontend y backend intercambian datos como **JSON**, un texto con estructura de objetos y listas. Es lo que devuelve el backend al listar tickets:

```
{ "id": 2, "title": "La impresora no funciona", "priority": "high",
  "status": "open", "author": { "id": 3, "name": "Luis" } }
```

CAPÍTULO 2

HTTP: pedir y responder

HTTP (HyperText Transfer Protocol) es el "idioma" con el que cliente y servidor se entienden. Es un protocolo de **petición y respuesta**: el cliente siempre empieza, el servidor nunca habla primero, y cada par petición/respuesta es independiente (el servidor no "recuerda" al cliente; por eso existe el JWT, capítulo 7).

Una petición real de HelpDesk

Cuando creas un ticket, Angular envía exactamente esto (puedes verlo en F12 → Network):

```
POST /api/tickets HTTP/1.1                      ← método + ruta + versión
Host: localhost:8000
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...    ← quién eres (JWT)
Content-Type: application/json                   ← el cuerpo es JSON

{"title":"La impresora no funciona","description":"Atasco","priority":"high"}
```

Y el servidor responde:

```
HTTP/1.1 201 Created                             ← código de estado
content-type: application/json
access-control-allow-origin: http://localhost:4200

{"id":2,"title":"La impresora no funciona","status":"open", ...}
```

Las cuatro partes de un mensaje HTTP

- **Línea inicial**: método y ruta (petición) o código de estado (respuesta).
- **Cabeceras (headers)**: metadatos como `Authorization`, `Content-Type` o `Origin`.
- **Línea en blanco**: separa cabeceras del cuerpo.

- **Cuerpo (body)**: los datos. Las peticiones GET y DELETE normalmente no llevan.

Códigos de estado que usa el proyecto

El primer dígito dice la familia: **2xx** éxito, **4xx** error del cliente, **5xx** error del servidor.

Código	Significado	Dónde ocurre en HelpDesk
200 OK	Todo bien, aquí están los datos.	Listar tickets, hacer login, actualizar un ticket.
201 Created	Se creó un recurso nuevo.	Registrarse y crear un ticket.
204 No Content	Hecho, y no hay nada que devolver.	Borrar un ticket.
400 Bad Request	La petición no tiene sentido.	Un admin intenta modificarse a sí mismo.
401 Unauthorized	No sabemos quién eres (sin token, vencido o inválido).	El interceptor cierra la sesión al recibirlo.
403 Forbidden	Sabemos quién eres, pero no tienes permiso.	Un `user` intenta cambiar el estado de un ticket.
404 Not Found	No existe ese recurso.	Ticket o usuario con un id inexistente.
409 Conflict	Choca con algo que ya existe.	Registrar un email ya usado.
422 Unprocessable	Los datos no pasan la validación.	Email mal formado o contraseña de menos de 8 caracteres.

> **401 contra 403: la confusión más común**
> **401** = "no te conozco" (autenticación). **403** = "te conozco, pero no puedes" (autorización). El frontend reacciona distinto: ante un 401 cierra sesión; ante un 403 solo avisa.

CAPÍTULO 3

Métodos HTTP: GET, POST, PUT, PATCH, DELETE

El **método** indica la *intención* de la petición. Junto con la ruta forma un patrón llamado **API REST**: las rutas son sustantivos (`/tickets`) y los métodos son los verbos (leer, crear, cambiar, borrar).

Método	Intención	¿Seguro?	¿Idempotente?	En HelpDesk
GET	Leer	Sí	Sí	`GET /tickets`, `GET /users`
POST	Crear / ejecutar acción	No	No	`POST /tickets`, `POST /auth/login`
PUT	Reemplazar completo	No	Sí	No se usa (ver abajo)
PATCH	Modificar parcialmente	No	Normalmente sí	`PATCH /tickets/1`, `PATCH /users/3`
DELETE	Borrar	No	Sí	`DELETE /tickets/1`

- **Seguro**: no cambia nada en el servidor. Por eso el navegador puede repetir un GET sin miedo.

- **Idempotente**: hacerlo 1 o 10 veces deja el servidor igual. Borrar el ticket 1 dos veces: el resultado final es el mismo. Crear un ticket dos veces: ¡dos tickets! Por eso POST no es idempotente y conviene deshabilitar el botón mientras envía.

Los cinco, con el código real

Todas las llamadas HTTP del frontend viven en un único archivo, `core/api.service.ts`:

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  private url = environment.apiUrl;              // http://localhost:8000/api

  tickets() {                                     // GET    → leer
    return this.http.get<Ticket[]>(`${this.url}/tickets`);
  }
  createTicket(data: { title: string; description: string; priority: Priority }) {
    return this.http.post<Ticket>(`${this.url}/tickets`, data);      // POST   → crear
  }
  updateTicketStatus(id: number, status: TicketStatus) {
    return this.http.patch<Ticket>(`${this.url}/tickets/${id}`, { status });  // PATCH
  }
  deleteTicket(id: number) {
    return this.http.delete<void>(`${this.url}/tickets/${id}`);      // DELETE → borrar
  }
}
```

PUT frente a PATCH

Imagina un ticket con título, descripción, prioridad y estado. Un agente solo quiere pasarlo a "cerrado":

```
PUT   /api/tickets/1   {"title":"...","description":"...","priority":"high","status":"closed"}
      → obliga a reenviar TODOS los campos; los que falten se pierden o se rechazan.

PATCH /api/tickets/1   {"status":"closed"}
      → solo envía lo que cambia. Es lo que usa HelpDesk.
```

Por eso el proyecto usa PATCH: un agente cambia el estado, un admin cambia el rol o la activación de un usuario, y nadie necesita reenviar el resto. Usa PUT cuando el cliente edita un recurso completo con un formulario.

El mismo endpoint, visto desde el servidor

```
@router.post("", response_model=TicketOut, status_code=201)   # POST /api/tickets → 201
def create_ticket(data: TicketIn, db: DB, user: CurrentUser, background: BackgroundTasks):
    ticket = Ticket(**data.model_dump(), author_id=user.id)    # el autor sale del TOKEN,
    db.add(ticket)                                             # no del cuerpo: no se puede
    db.commit()                                                # suplantar a otra persona
    ...
```

> **Tu turno: probar los métodos sin Angular**
> Con el backend corriendo, abre http://localhost:8000/docs. Es una interfaz generada automáticamente donde puedes ejecutar cada endpoint, ver los códigos de estado y entender el contrato antes de escribir una sola línea de Angular.

CAPÍTULO 4

HTML y el DOM

HTML es el texto que describe la estructura de una página. Cuando el navegador lo lee, construye en memoria un **árbol de objetos** llamado **DOM** (Document Object Model). JavaScript no modifica el texto HTML: modifica ese árbol, y el navegador repinta lo que cambió.

```
HTML (texto)                    DOM (árbol en memoria)
<main>                          main
  <section id="grid">            └─ section#grid
    <article class="card">            ├─ article.card
      <h3>Camiseta</h3>               │    └─ h3 "Camiseta"
    </article>                        └─ article.card ...
```

Manipular el DOM a mano: StoreAPI

El proyecto StoreAPI usa JavaScript puro, así que ahí se ve el DOM sin ayuda de ningún framework. Esta función crea elementos de forma segura:

```
// StoreAPI/js/main.js
function h(tag, props = {}, ...children) {
    const el = document.createElement(tag);          // 1. crear nodo
    for (const [key, value] of Object.entries(props)) {
        if (key === "class") el.className = value;   // 2. configurar atributos
        else if (key.startsWith("on")) el.addEventListener(key.slice(2), value);
        else el.setAttribute(key, value);
    }
    el.append(...children);                          // 3. colgarle hijos (texto o nodos)
    return el;
}

els.grid.replaceChildren(...list.map(productCard));  // 4. reemplazar el contenido
```

- `document.getElementById('grid')` / `querySelector('.card')`: buscan nodos.
- `createElement` + `append`: construyen nodos nuevos.
- `addEventListener('click', ...)`: reaccionan a lo que hace el usuario.
- `replaceChildren`: reemplaza todo el contenido de un nodo de una vez.

Eventos y delegación

Cada vez que hay una acción del usuario (clic, teclear, enviar un formulario) el navegador dispara un **evento**. Con 20 productos, ¿ponemos 20 escuchadores de clic? No: se pone **uno solo** en el contenedor, y se averigua qué botón se pulsó. Es la **delegación de eventos**:

```
els.grid.addEventListener("click", (e) => {
    const add = e.target.closest("[data-add]");      // ¿el clic fue en un botón "Agregar"?
    if (add) return addToCart(Number(add.dataset.add));
    const open = e.target.closest("[data-open]");    // ¿o en un producto para ver detalle?
    if (open) openProduct(Number(open.dataset.open));
});
```

Seguridad: `textContent` frente a `innerHTML`

> **XSS: el ataque que evita esta decisión**
> Si un título de producto (o la descripción de un ticket) contiene `<script>alert(1)</script>` y lo insertas con `innerHTML`, el navegador **ejecuta ese código**. Con `textContent` (que usa `h()`) se muestra como texto inofensivo. Angular hace esto mismo por ti: `{{ t.title }}` siempre escapa el contenido. En HelpDesk lo comprobamos: un ticket con un `<script>` aparece escapado también en el correo.

En Angular el DOM lo gestiona el framework

En el frontend de HelpDesk casi nunca llamas a `createElement`. Describes cómo debe verse la pantalla según los datos (la **plantilla**) y Angular actualiza el DOM por ti cuando los datos cambian. Lo verás en el capítulo 6.

CAPÍTULO 5

CSS: los estilos

HTML dice *qué* es cada cosa; **CSS** dice *cómo se ve*. Una regla CSS tiene un **selector** (a quién) y declaraciones (qué cambiar):

```
.chip {                       /* selector: todo elemento con class="chip" */
    padding: 8px 16px;         /* propiedad: valor; */
    border-radius: 999px;      /* esquinas totalmente redondeadas */
}
.chip[aria-pressed="true"] {   /* selector de atributo: solo el chip activo */
    background: var(--ink);
    color: #fff;
}
```

Selectores más usados

Selector	Elige	Ejemplo en el proyecto
`p`	Por etiqueta	`button { cursor: pointer; }`
`.clase`	Por clase	`.card`, `.badge`, `.error`
`#id`	Por id (único)	Poco usado: tiene demasiado peso
`a b`	Descendiente	`header a` (enlaces dentro del header)
`a:hover`	Estado del ratón	`.card:hover` eleva la tarjeta
`[attr]`	Por atributo	`.cart-count[hidden]`

La cascada y la especificidad

"Cascading" significa que varias reglas pueden aplicar al mismo elemento y hay un orden para decidir quién gana: 1) la más **específica** (un id pesa más que una clase y una clase más que una etiqueta) y 2) a igual especificidad, la que viene **después**.

> **Un bug real que ocurrió en este proyecto**
> En StoreAPI el contador del carrito se veía como un punto naranja aunque estuviera vacío. El HTML tenía `hidden` (que el navegador traduce a `display:none`), pero mi regla `.cart-count { display: grid }` tenía más prioridad y lo anulaba. La solución fue una regla más específica: `.cart-count[hidden] { display: none; }`.

El modelo de caja

Todo elemento es una caja con cuatro capas, de dentro hacia fuera: **contenido** → **padding** → **borde** → **margin**. Por eso el proyecto empieza con `* { box-sizing: border-box; }`: así el ancho incluye padding y borde y las cuentas salen como esperas.

Maquetación moderna: Flexbox y Grid

```
/* Flexbox: una dimensión (fila o columna). Ideal para barras y grupos de botones. */
header { display: flex; align-items: center; gap: 24px; }

/* Grid: dos dimensiones. Aquí: tantas columnas de mínimo 240px como quepan. */
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(240px, 1fr)); gap: 24px; }
```

Esa única línea del `.grid` hace que la tienda tenga 4 columnas en escritorio, 2 en tablet y 1 en móvil pequeño, sin escribir un solo cálculo.

Variables CSS y diseño adaptable

```
:root {                                 /* se define una vez ... */
    --accent: #e4572e;
    --radius: 16px;
}
.btn.primary { background: var(--accent); }   /* ... y se usa en todos lados */

@media (max-width: 680px) {             /* media query: reglas solo en pantallas pequeñas */
    .grid { grid-template-columns: repeat(2, 1fr); }
}
```

Cambiar `--accent` recolorea toda la tienda. Y con la media query la página se adapta al móvil: eso es **diseño responsivo**.

Estilos en Angular

En HelpDesk hay dos niveles: `src/styles.css` (estilos **globales**, como `.card` o `.badge`) y `app.css` (estilos **del componente**). Angular encapsula los de componente para que no se filtren al resto de la página.

CAPÍTULO 6

Angular paso a paso

Angular es un framework para construir la parte visible de una aplicación (el frontend). Resuelve lo que en StoreAPI hacíamos a mano: mantener el DOM sincronizado con los datos, organizar el código y navegar entre pantallas sin recargar.

Estructura del proyecto

```
frontend/src/
├── main.ts                      ← arranque: bootstrapApplication(App, appConfig)
├── styles.css                   ← estilos globales (compartidos)
├── environments/environment.ts  ← apiUrl: 'http://localhost:8000/api'
└── app/
    ├── app.ts / .html / .css / .spec.ts   ← componente raíz: barra de navegación + <router-outlet>
    ├── app.config.ts            ← configuración: router, HttpClient, interceptor
    ├── app.routes.ts            ← mapa de rutas (qué URL muestra qué pantalla)
    ├── core/                    ← lógica compartida (cada archivo con su .spec.ts)
    │   ├── auth.service.ts      ← sesión, login, registro, token
    │   ├── api.service.ts       ← TODAS las llamadas HTTP de tickets y usuarios
    │   ├── auth-interceptor.ts  ← añade el JWT a cada petición
    │   ├── auth-guard.ts        ← protege rutas: exige sesión iniciada
    │   ├── role-guard.ts        ← protege rutas: exige un rol concreto
    │   └── models.ts            ← tipos: User, Ticket, Role ...
    └── pages/                   ← una carpeta por pantalla
        ├── login/    login.ts  login.html  login.css  login.spec.ts
        ├── register/ register.ts ... (los mismos 4 archivos)
        ├── tickets/  tickets.ts ...
        └── users/    users.ts ...
```

1. Componentes: cuatro archivos por pantalla

Un **componente** es un trozo de interfaz reutilizable. Angular lo separa en cuatro archivos con el mismo nombre, cada uno con una sola responsabilidad. No se escriben a mano: los crea el CLI.

```
ng generate component pages/login        # crea la carpeta pages/login/ con:

login.ts        ← la CLASE: datos y comportamiento (TypeScript)
login.html      ← la PLANTILLA: qué se dibuja (HTML + sintaxis de Angular)
login.css       ← los ESTILOS: solo de este componente (Angular los encapsula)
login.spec.ts   ← las PRUEBAS automáticas del componente
```

El decorador `@Component` conecta los tres primeros. Resumen de `pages/login/login.ts`:

```
@Component({
  imports: [FormsModule, RouterLink],     // lo que la plantilla puede usar
  selector: 'app-login',                  // etiqueta HTML del componente
  styleUrl: './login.css',                // → los estilos
  templateUrl: './login.html',            // → la plantilla
})
export class Login {
  email = '';
  password = '';
  loading = signal(false);
  error = signal('');
  submit() { ... }
}
```

Y un extracto de `pages/login/login.html`, donde viven los enlaces con la clase:

```
<form (ngSubmit)="submit()">                        <!-- evento: al enviar, llama a submit() -->
  <input type="email" name="email" [(ngModel)]="email" required />   <!-- enlace de datos -->
```

```
  @if (error()) { <p class="error">{{ error() }}</p> }              <!-- condicional -->
  <button [disabled]="loading()">Entrar</button>
</form>
```

Los estilos que solo usa esta pantalla van en `login.css` (por ejemplo, el ancho de la tarjeta). Lo que comparten todas las pantallas (botones, tarjetas, tablas) queda en el `styles.css` global.

2. Enlace de datos (data binding)

Sintaxis	Dirección	Qué hace	Ejemplo
`{{ valor }}`	Clase → vista	Muestra un dato como texto (escapado).	`{{ t.title }}`
`[propiedad]="x"`	Clase → vista	Asigna una propiedad del elemento.	`[disabled]="loading()"`
`(evento)="f()"`	Vista → clase	Reacciona a un evento del DOM.	`(click)="auth.logout()"`
`[(ngModel)]="x"`	Ambas	Sincroniza un campo de formulario con una variable.	`[(ngModel)]="email"`

3. Control de flujo en plantillas: `@if` y `@for`

```
@if (auth.hasRole('admin')) {                <!-- solo el admin ve el enlace -->
  <a routerLink="/users">Usuarios</a>
}

@for (t of tickets(); track t.id) {          <!-- repite la fila por cada ticket -->
  <tr><td>{{ t.title }}</td> ...</tr>
} @empty {
  <tr><td colspan="7">No hay tickets.</td></tr>
}
```

`track t.id` le dice a Angular cómo identificar cada fila, así al cambiar la lista solo toca las filas afectadas en el DOM en lugar de rehacer todo.

4. Signals: datos que avisan cuando cambian

Un **signal** es una caja que guarda un valor y notifica a la plantilla cuando cambia. Se lee como una función (`tickets()`) y se modifica con `set` o `update`:

```
tickets = signal<Ticket[]>([]);

this.tickets.set(listaDelServidor);                                  // reemplaza
this.tickets.update((list) => [nuevoTicket, ...list]);               // al crear: lo agrega primero
this.tickets.update((list) => list.filter((x) => x.id !== t.id));    // al borrar: lo quita

readonly isLoggedIn = computed(() => !!this.token() && !!this.user()); // derivado automático
```

Cuando haces `update`, Angular repinta únicamente lo que depende de ese signal. Ese es el equivalente automático de lo que hacía `replaceChildren` en StoreAPI.

5. Servicios e inyección de dependencias

La lógica que no pertenece a una pantalla concreta (sesión, llamadas HTTP) va en **servicios**. Un componente los pide con `inject()` y Angular le entrega **una única instancia compartida** (`providedIn: 'root'`):

```
export class Tickets {
```

```
  private api = inject(ApiService);        // "dame el ApiService"
  protected auth = inject(AuthService);    // "dame el AuthService": es el mismo en toda la app
  ...
```

Gracias a eso, cuando `Login` guarda el usuario en `AuthService`, la barra de navegación (otro componente) lo ve al instante.

6. HttpClient y Observables

Las llamadas de red son **asíncronas**: la respuesta tarda y la pantalla no puede congelarse esperando. `HttpClient` devuelve un **Observable**: una promesa de datos futuros. **Nada se envía hasta que alguien hace** `subscribe`:

```
create() {
  this.api.createTicket({ title: this.title, description: this.description,
                          priority: this.priority })
    .subscribe({
      next:  (t)  => this.tickets.update((list) => [t, ...list]),   // llegó 201
      error: ()   => this.error.set('No se pudo crear el ticket'),  // llegó 4xx/5xx
    });
}
```

7. El enrutador y la carga diferida

```
export const routes: Routes = [
  { path: 'login',   loadComponent: () => import('./pages/login/login').then((m) => m.Login) },
  { path: 'tickets', canActivate: [authGuard],
                    loadComponent: () => import('./pages/tickets/tickets').then((m) => m.Tickets) },
  { path: 'users',   canActivate: [roleGuard('admin')],
                    loadComponent: () => import('./pages/users/users').then((m) => m.Users) },
  { path: '**', redirectTo: 'tickets' },
];
```

- **Router**: cambia la "pantalla" según la URL, sin recargar la página (es una *SPA*, Single Page Application).
- **loadComponent con import()**: el código de cada pantalla se descarga solo cuando entras (*lazy loading*). Por eso al compilar vimos *chunks* separados: login, tickets, users.
- **canActivate**: los *guards* deciden si puedes entrar. Si no, te redirigen.

8. Guards

```
// core/auth-guard.ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.isLoggedIn() || inject(Router).createUrlTree(['/login']);  // true, o redirige
};

// core/role-guard.ts  (una "fábrica": recibe los roles permitidos)
export const roleGuard = (...roles: Role[]): CanActivateFn => () => { ... };
```

> **Los guards NO son seguridad**
> Un guard solo mejora la experiencia: oculta pantallas a quien no corresponden. Cualquier usuario puede modificar el JavaScript de su navegador o llamar a la API directamente. **La seguridad real está en el backend**, que valida el token y el rol en cada endpoint (capítulo 7).

9. Pruebas automáticas: el archivo `.spec.ts`

Cada pieza tiene junto a ella su `.spec.ts` (lo genera el CLI). Son programas que ejecutan tu código y comprueban que hace lo esperado, sin abrir el navegador ni llamar al backend real. HelpDesk las ejecuta con **Vitest**:

```
ng test                    # modo interactivo, se repite al guardar
ng test --watch=false      # una sola pasada (útil antes de entregar)
```

La clave para probar HTTP es `HttpTestingController`: intercepta las peticiones, te deja revisarlas (método, URL, cuerpo) y tú decides qué respuesta simular. Así se prueba el PATCH del cambio de estado sin servidor:

```
it('setStatus() sends a PATCH and updates the row', () => {
  component.setStatus(ticket, 'closed');

  const req = http.expectOne(`${url}/1`);              // ¿salió UNA petición a esta URL?
  expect(req.request.method).toBe('PATCH');            // ¿con el método correcto?
  expect(req.request.body).toEqual({ status: 'closed' });  // ¿y solo con lo que cambia?

  req.flush({ ...ticket, status: 'closed' });          // simulamos la respuesta 200
  expect(component.tickets()[0].status).toBe('closed');    // la pantalla se actualizó
});
```

Las pruebas de HelpDesk también protegen reglas de negocio: que un `user` no vea el botón de borrar, que el interceptor cierre sesión con un 401 pero **no** con un 403, o que un JWT vencido se descarte al abrir la app.

CAPÍTULO 7

Autenticación: JWT, interceptor y roles

Como HTTP no recuerda quién eres entre una petición y la siguiente, hay que demostrar tu identidad **en cada una**. Para eso el servidor te entrega, al iniciar sesión, un **JWT** (JSON Web Token): un carnet firmado.

Anatomía de un JWT

```
eyJhbGciOiJIUzI1NiIs...  .  eyJzdWIiOiIzIiwiZXhwIjoxNzkx...  .  Uod30wg4aPepQg9Q...
└──── cabecera ──────┘     └──────── contenido (payload) ──────┘     └──── firma ────┘
{ "alg": "HS256" }            { "sub": "3", "exp": 1791649468 }        HMAC con SECRET_KEY
```

- `sub`: el id del usuario. `exp`: cuándo vence.
- La **firma** se calcula con una clave que solo conoce el servidor. Si alguien cambia el contenido, la firma deja de coincidir y el servidor lo rechaza.
- El contenido está codificado, **no cifrado**: cualquiera puede leerlo. Nunca pongas contraseñas ahí.

El flujo completo

```
1. POST /auth/login {email, password}      →  servidor verifica con bcrypt
2. 200 {access_token, user}                ←  servidor firma un JWT
3. Angular guarda el token (localStorage) y el usuario (signal)
4. Cada petición siguiente lleva:  Authorization: Bearer <token>
5. FastAPI valida firma + vencimiento, busca al usuario, y comprueba su rol
```

El interceptor: añadir el token una sola vez

En lugar de acordarse de poner la cabecera en cada llamada, un **interceptor** se interpone en *todas* las peticiones salientes:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.token();
  if (token) req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });

  return next(req).pipe(
    catchError((err) => {
      if (err.status === 401 && token) auth.logout();   // token vencido: cerrar sesión
      return throwError(() => err);
    }),
  );
};
```

Roles: dos niveles de protección

Rol	Puede	Capa que lo impide
user	Crear tickets y ver solo los suyos.	Backend: `if user.role == Role.user: query.where(author_id == user.id)`
agent	Ver todos los tickets y cambiar su estado.	Backend: `require_roles(agent, admin)` → 403 si no
admin	Todo, incluido borrar tickets y gestionar usuarios.	Backend: `require_roles(admin)`

```
# backend/app/deps.py
def require_roles(*roles: Role):
    def checker(user: CurrentUser) -> User:
        if user.role not in roles:
            raise HTTPException(403, "No tienes permisos para esta acción")
        return user
    return checker
```

Detalle de diseño: el JWT guarda **solo el id**, no el rol. En cada petición el servidor lee el rol actual de la base de datos. Así, si un admin degrada o desactiva a alguien, el cambio surte efecto de inmediato en lugar de esperar a que venza el token.

CORS: por qué aparecen peticiones OPTIONS

El navegador aplica la **política del mismo origen**: una página en `localhost:4200` no puede leer respuestas de `localhost:8000` a menos que ese servidor lo autorice (puerto distinto = origen distinto). Para una petición con cabecera `Authorization`, el navegador primero envía por su cuenta un **preflight**:

```
OPTIONS /api/tickets                        ← "¿me dejas hacer esto desde 4200?"
Origin: http://localhost:4200
Access-Control-Request-Headers: authorization

200 OK
access-control-allow-origin: http://localhost:4200    ← "sí"  → ahora sí viaja el GET/POST real
```

Esto lo vimos en el log del servidor: cada acción del frontend aparece precedida de un `OPTIONS`. Se configura en `main.py` con `CORSMiddleware` y la variable `CORS_ORIGINS`.

CAPÍTULO 8

Todo junto: la vida de un clic

Sigamos exactamente qué ocurre cuando un usuario pulsa **"Crear ticket"**. Cada paso usa un concepto de los capítulos anteriores.

#	Qué pasa	Concepto
1	El usuario rellena el formulario. `[(ngModel)]` copia cada tecla a las variables `title` y `description`.	Enlace de datos, DOM
2	Pulsa el botón: el navegador lanza el evento `submit`; Angular ejecuta `create()`.	Eventos
3	`ApiService.createTicket()` prepara un `POST`. Aún no se envía.	Observable, método POST
4	`.subscribe()` lo dispara. El interceptor añade `Authorization: Bearer ...`.	Interceptor, JWT
5	El navegador envía un `OPTIONS` (preflight) y, si lo aprueban, el `POST` real.	HTTP, CORS
6	FastAPI valida el token, busca al usuario y comprueba que está activo.	401 si falla
7	Pydantic valida el cuerpo (`TicketIn`): título no vacío, prioridad válida.	422 si falla
8	Se guarda el ticket en PostgreSQL y se encolan los correos (autor + agentes) en segundo plano.	Base de datos, Mailpit
9	El servidor responde `201 Created` con el ticket en JSON.	Código de estado
10	`next` se ejecuta: `tickets.update(...)` añade el ticket al signal.	Signals
11	Angular detecta el cambio, ejecuta de nuevo el `@for` y actualiza solo las filas necesarias del DOM.	DOM, plantillas
12	El navegador repinta la pantalla aplicando las reglas CSS. El usuario ve su ticket en la lista.	CSS

Cómo verlo con tus propios ojos

- Abre HelpDesk, pulsa **F12** y ve a la pestaña **Network**.
- Crea un ticket y localiza el `POST tickets`. Revisa **Headers** (verás el Bearer), **Payload** (el JSON enviado) y **Response** (el 201).
- En **Elements** inspecciona la fila nueva: es el DOM real que Angular generó.
- Abre http://localhost:8025 (Mailpit): ahí están los correos que disparó ese mismo POST.

CAPÍTULO 9

Ejercicios y glosario

Ejercicios sugeridos (de menor a mayor dificultad)

- **GET con filtros**: añade en la pantalla de tickets un `<select>` para filtrar por estado. Primero hazlo en el cliente con un `computed`; luego pásalo al servidor con `?status=open`.
- **CSS**: en HelpDesk, cambia el color de acento por una variable CSS (`:root { --accent }`) como en StoreAPI y úsala en botones y enlaces.
- **DELETE**: añade el botón "Borrar" también para el autor de un ticket que siga abierto. Decide en el backend qué código devuelves si un usuario intenta borrar el de otro (¿403 o 404?).
- **PUT**: crea `PUT /tickets/{id}` para que un usuario edite título y descripción de su ticket abierto, con formulario en Angular. Compara con el PATCH.
- **Interceptor**: crea otro con `ng generate interceptor core/loading` que muestre un indicador de "cargando" mientras haya peticiones en curso, y escribe su `.spec.ts`.
- **Seguridad**: desde la consola del navegador, intenta llamar a `DELETE /api/tickets/1` con el token de un usuario normal y verifica que el servidor responde 403 aunque el botón esté oculto.

Glosario rápido

Término	Significado
API	Contrato de rutas y datos por el que dos programas se comunican.
REST	Estilo de API: rutas como sustantivos y métodos HTTP como verbos.
Endpoint	Una combinación concreta de método y ruta, por ejemplo `POST /api/tickets`.
DOM	Árbol en memoria que representa la página; JavaScript lo manipula.
SPA	Aplicación de una sola página: cambia pantallas sin recargar el navegador.
Componente	Pieza de interfaz = clase TypeScript + plantilla HTML (+ estilos).
Signal	Valor reactivo que avisa a la plantilla cuando cambia.
Observable	Flujo de datos asíncrono; no se ejecuta hasta hacer `subscribe`.
Interceptor	Código que se ejecuta en cada petición o respuesta HTTP.
Guard	Función que decide si una ruta puede activarse.
JWT	Token firmado que prueba la identidad en cada petición.
CORS	Mecanismo por el que un servidor autoriza a páginas de otro origen.
Idempotente	Repetir la operación no cambia el resultado final.
XSS	Inyección de código a través de contenido no escapado.
Preflight	Petición OPTIONS que el navegador envía antes de una petición CORS.

Siguiente paso
Cambia algo pequeño, rompe algo a propósito y mira qué dice la pestaña Network y la consola. Entender los errores (un 401, un 422, un fallo de CORS) enseña más que cualquier lectura.